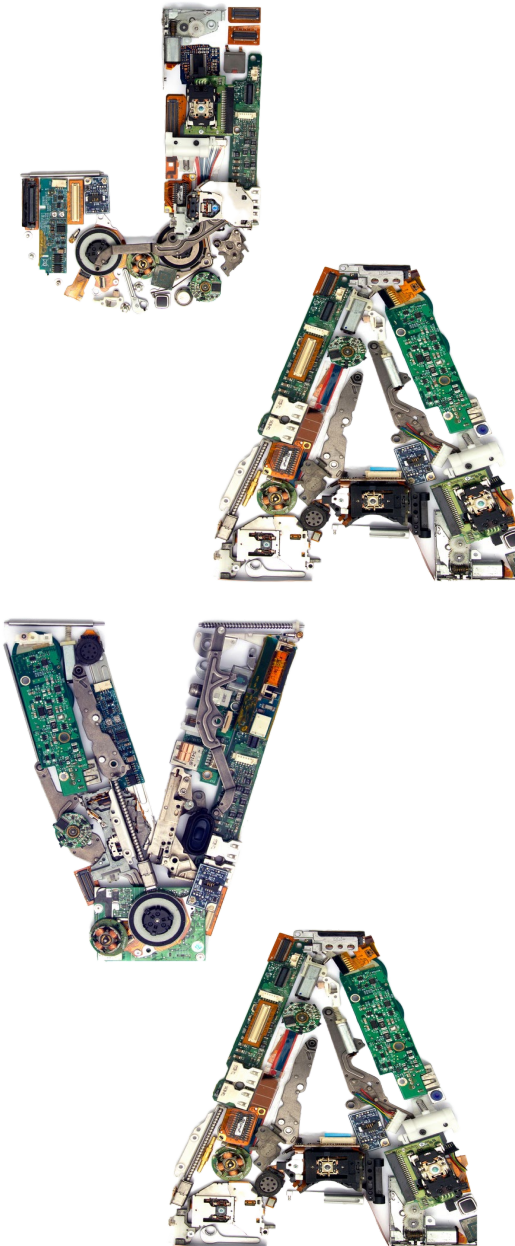


Programação Orientada a Objetos com Java e WEB

Organização de projeto com Maven e JDBC



Agenda

- A classe de conexão: da versão original à **ConnectionFactory**
- O construtor **private** e o papel das classes utilitárias
- Arquivos **.properties**: separando configuração de código
- A interface **GenericDAO**: contrato comum entre os DAOs
- A classe **Optional**: ausência de valor sem *null*

Fio condutor da aula: cada decisão de projeto é avaliada pelas vantagens e pelas desvantagens que traz.

Classe de conexão – *a versão original*

```
public class Conexao {  
    private final String url =  
        "jdbc:oracle:thin:@oracle.fiap.com.br:1521:ORCL";  
    private final String user = "";  
    private final String password = "";  
  
    public Connection conectar() {  
        try {  
            return DriverManager.getConnection(  
                url, user, password);  
        } catch (SQLException e) {  
            System.out.println(e.getMessage());  
        }  
        return null;  
    }  
}
```

Classe de conexão – *três problemas*

- **Instanciação repetida:** cada DAO executa *new Conexao()*, criando um objeto cujo único propósito é chamar um método.
- **Falha silenciosa:** em caso de erro o método devolve *null*. O DAO só descobre o problema depois, como *NullPointerException*.
- **Credenciais no código-fonte:** usuário e senha ficam compilados na classe e vão para o repositório de versionamento.

Os três problemas têm a mesma origem: a classe acumula configuração, criação e tratamento de erro.

Classe de conexão – *ConnectionFactory*

```
public final class ConnectionFactory {  
    private static final Properties PROPS =  
        carregarPropriedades();  
  
    private ConnectionFactory() { }    // impede a instanciação  
  
    public static Connection obterConexao()  
        throws SQLException {  
        return DriverManager.getConnection(  
            PROPS.getProperty("db.url"),  
            PROPS.getProperty("db.user"),  
            PROPS.getProperty("db.password"));  
    }  
}
```

O método declara throws SQLException: quem chama decide.

Leitura da configuração

```
private static Properties carregarPropriedades() {
    Properties props = new Properties();
    try (InputStream in = ConnectionFactory.class
        .getClassLoader()
        .getResourceAsStream("database.properties")) {
        if (in == null) {
            throw new IllegalStateException(
                "Arquivo não encontrado.");
        }
        props.load(in);
        return props;
    } catch (IOException e) {
        throw new IllegalStateException("Falha na leitura.", e);
    }
}
```

Por que o construtor deve ser *private*

A classe não possui estado a ser mantido: todos os seus membros são estáticos. Instanciar *ConnectionFactory* não traria nenhum benefício, e um objeto vazio dessa classe seria apenas ruído no projeto.

Ao declarar o construtor como *private*, o compilador passa a impedir *new ConnectionFactory()*. A restrição deixa de ser uma recomendação escrita na documentação e passa a ser verificada em tempo de compilação.

- **Comunica a intenção:** a classe é utilitária, usada apenas pelo nome.
- **Impede o construtor implícito:** sem declaração, o Java criaria um construtor público automaticamente.
- **Combina com final:** a classe também não pode ser estendida.

Comparação entre as *implementações*

Versão original

- Instanciada em cada DAO
- Credenciais compiladas na classe
- Devolve null quando falha
- Erro aparece longe da causa
- Trocar de banco exige recompilar

ConnectionFactory

- Método estático, sem instância
- Credenciais em arquivo externo
- Propaga SQLException
- Erro aparece na origem
- Trocar de banco altera o arquivo

A nova versão não é mais curta: é mais previsível.

ConnectionFactory – *avaliação*

Vantagens

- Ponto único de configuração
- Nenhum DAO conhece a URL do banco
- Falha explícita, nunca silenciosa
- Facilita testes com outro banco

Desvantagens

- Membros estáticos dificultam a substituição em teste
- Uma conexão nova por operação tem custo
- Não gerencia pool de conexões

Em produção, o passo seguinte é um pool de conexões.

O arquivo *.properties*

É um arquivo de texto simples que armazena pares **chave = valor**. O Java oferece suporte nativo a esse formato por meio da classe *java.util.Properties*, sem necessidade de qualquer biblioteca externa.

Sua função é separar **configuração** de **código**: informações que mudam conforme o ambiente de execução ficam fora das classes compiladas.

- Localização no projeto Maven: **src/main/resources**
- Comentários começam por **#**; não se usam aspas nos valores
- Convenção de nomes com prefixo: **db.url**, **db.user**, **db.password**

O arquivo *.properties* – exemplo

```
# src/main/resources/database.properties
```

```
db.url=jdbc:oracle:thin:@oracle.fiap.com.br:1521:ORCL
db.user=rm000000
db.password=ddmmaa
```

```
Properties props = new Properties();
props.load(in);
```

```
String url = props.getProperty("db.url");
```

```
// segundo argumento: valor padrão se a chave não existir
String porta = props.getProperty("db.porta", "1521");
```

O arquivo *.properties* – avaliação

Vantagens

- Alterar o banco sem recompilar
- Um arquivo por ambiente
- Credenciais fora do código-fonte
- Suporte nativo da linguagem

Cuidados

- O arquivo continua legível: não é criptografia
- Deve ser excluído do versionamento
- Erro de digitação na chave devolve null

Incluir `database.properties` no `.gitignore` e versionar apenas um modelo, sem as senhas reais.

A interface *GenericDAO*

É um **contrato** que declara as operações comuns a todos os DAOs do projeto. Utiliza dois parâmetros de tipo: **T** para a entidade manipulada e **ID** para o tipo da chave primária.

Seu objetivo não é reaproveitar código — cada DAO escreve o próprio SQL — mas **padronizar nomes e assinaturas**, de modo que o compilador cobre a uniformidade que antes dependia da disciplina de cada integrante da equipe.

Sem contrato: listarTodos(), buscarTodos() e getAll()
convivendo no mesmo projeto.

A interface *GenericDAO* – o código

```
public interface GenericDAO<T, ID> {  
    void inserir(T entidade);  
    List<T> listarTodos();  
    Optional<T> buscarPorId(ID id);  
}
```

```
public class CategoriaDAO  
    implements GenericDAO<Categoria, Integer> {  
  
    @Override  
    public void inserir(Categoria categoria) { /* ... */ }  
  
    // método exclusivo deste DAO: fora do contrato  
    public List<Categoria> listarPorNome(String nome) {  
        /* ... */  
    }  
}
```

A interface *GenericDAO* – avaliação

Vantagens

- Nomes uniformes em todos os DAOs
- O compilador cobra a padronização
- @Override documenta a origem
- Permite tratar DAOs pelo tipo comum

Desvantagens

- Obriga todas as classes a implementar tudo
- Métodos vazios quando a operação não se aplica
- Pelo tipo da interface, os métodos específicos ficam inacessíveis

Manter no contrato apenas o que é realmente comum.

A classe *Optional*

Introduzida no Java 8, *Optional<T>* é um invólucro que representa um valor que **pode ou não existir**. Possui exatamente dois estados possíveis, e nenhum outro:

```
// contém um valor
```

```
Optional<Categoria> presente = Optional.of(categoria);
```

```
// não contém valor algum
```

```
Optional<Categoria> vazio = Optional.empty();
```

Aplicação típica: retorno de consultas que podem não encontrar registro algum.

Optional *no DAO*

```
public Optional<Categoria> buscarPorId(Integer id) {  
    String sql = "select id, categoria"  
                + " from java_categoria where id = ?";  
    try (Connection conn = ConnectionFactory.obterConexao();  
        PreparedStatement ps = conn.prepareStatement(sql)) {  
        ps.setInt(1, id);  
        try (ResultSet rs = ps.executeQuery()) {  
            if (rs.next()) {  
                return Optional.of(montarCategoria(rs));  
            }  
            return Optional.empty(); // ausência esperada  
        }  
    } catch (SQLException e) {  
        throw new DBException("Erro ao buscar.", e);  
    }  
}
```

Optional *no consumo*

```
Optional<Categoria> resultado = dao.buscarPorId(id);

if (resultado.isPresent()) {
    JOptionPane.showMessageDialog(null,
        resultado.get().getCategoria());
} else {
    JOptionPane.showMessageDialog(null, "Não encontrada.");
}
```

```
// forma equivalente, sem ramificação explícita
String texto = dao.buscarPorId(id)
    .map(Categoria::getCategoria)
    .orElse("Categoria não encontrada");
```

Optional ou *return null*?

Com null

```
Categoria c =  
    dao.buscarPorId(99);
```

```
c.getCategoria();
```

```
// compila sem aviso  
// falha em execução
```

Com Optional

```
Optional<Categoria> c =  
    dao.buscarPorId(99);
```

```
c.getCategoria();
```

```
// ERRO DE COMPILAÇÃO  
// o método não existe
```

O ganho é antecipar o erro do tempo de execução
para o tempo de compilação.

Por que evitar o *null*

- **Erro antecipado:** o compilador impede o acesso direto ao valor; com *null*, o programa compila e falha apenas em execução.
- **Assinatura como documentação:** *Optional<Categoria> buscarPorId* declara a possibilidade de ausência; *Categoria buscarPorId* nada informa.
- **Distinção entre casos:** ausência esperada é *Optional.empty()*; falha de acesso ao banco é uma exceção. Com *null*, as duas se confundem.

Optional não substitui a disciplina do programador:
transfere essa disciplina para o compilador.

Optional – *cuidados no uso*

- Utilizar somente como **tipo de retorno** de método: nunca como atributo de classe nem como parâmetro.
- Não envolver coleções: uma *List* vazia já expressa a ausência de resultados.
- Evitar *get()* sem verificação prévia — lança *NoSuchElementException* e reproduz o problema original.

Preferir sempre *orElse*, *orElseThrow*, *ifPresent* ou *map*.

Síntese da *aula*

- A conexão pertence a uma **única classe utilitária**, com construtor privado e método estático.
- A **configuração** fica em arquivo externo, fora do controle de versão.
- O **contrato comum** entre DAOs padroniza nomes, mas deve conter apenas o que é efetivamente compartilhado.
- O retorno de consultas por identificador usa **Optional**, tornando a ausência de resultado visível na assinatura.

Cada decisão apresentada tem custo: o critério é escolher o custo que se paga uma vez, e não a cada manutenção.